

BUGFIX_AI

AI-Powered Autonomous Bug Fixing Assistant

By Kavya Loona
26A115

Problem Statement

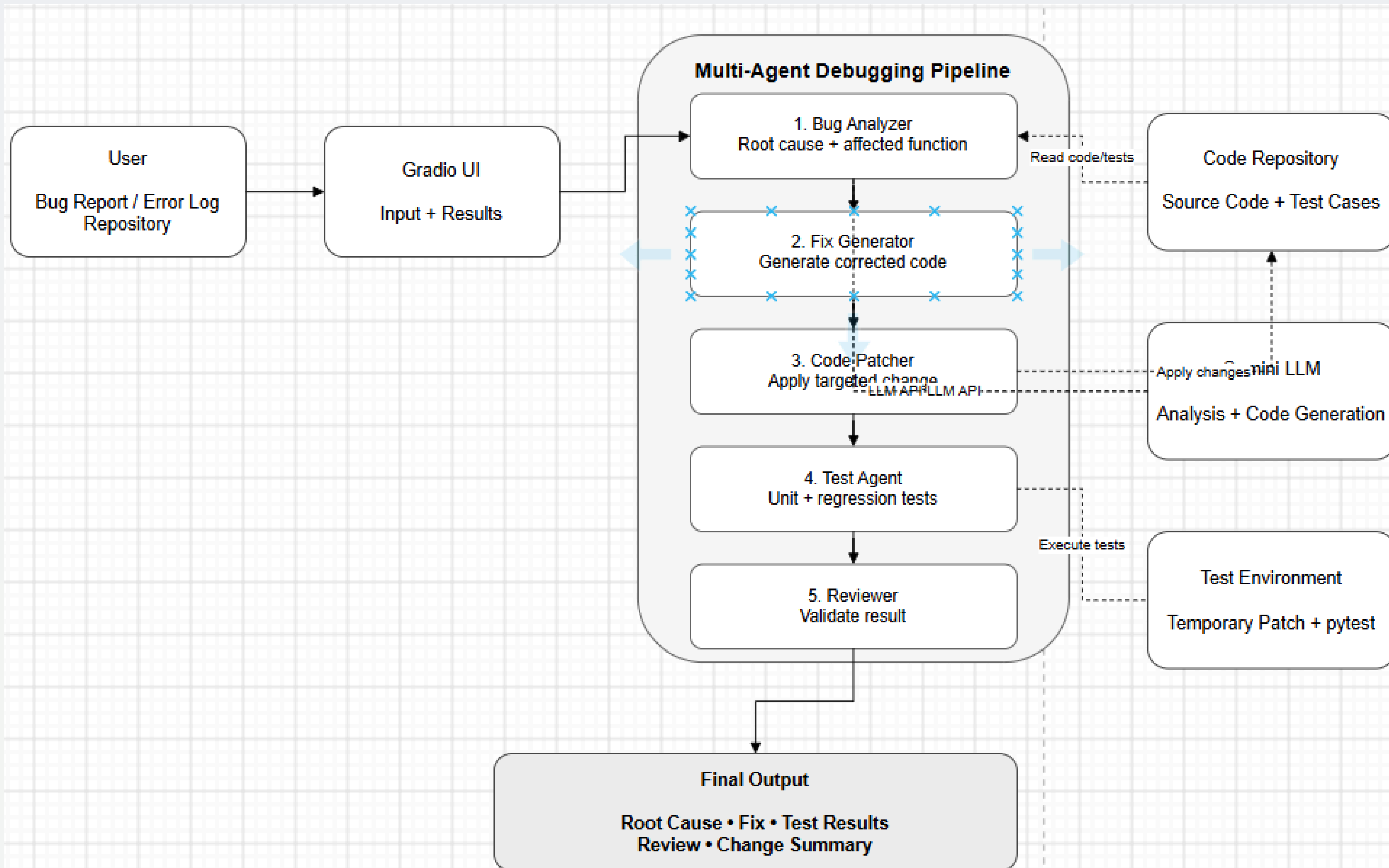
Software debugging usually takes several manual steps. A developer has to understand the bug, locate the affected code, identify the root cause, make a change, run tests, and then check whether the change has affected anything else.

This becomes time-consuming when the codebase has multiple files, dependencies, and test cases. The objective of this project is to build an autonomous debugging system that can take a bug report, understand the existing code, generate a targeted fix, test it, and provide a clear summary of what was changed.

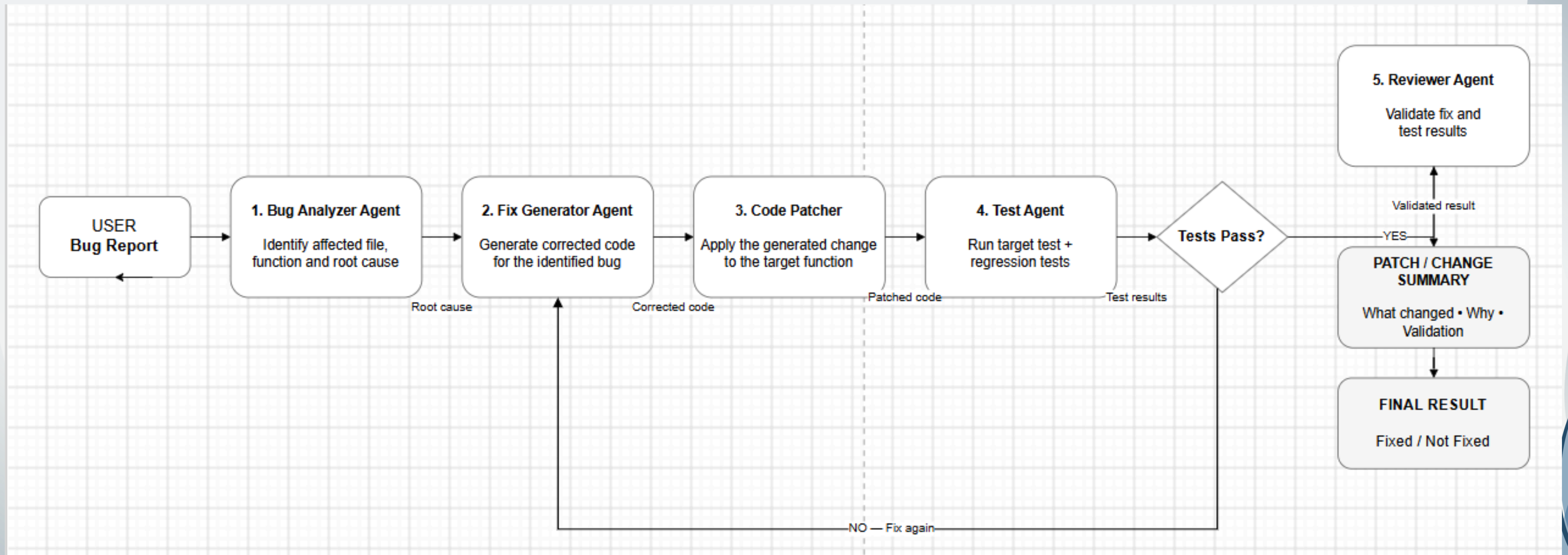
Goal

Reduce the amount of manual effort required between “a bug is reported” and “a validated fix is available.”

System Architecture



Multi-Agent Workflow



Our Solution

From Bug Report to Validated Fix

The system takes a simple bug report as input instead of expecting the user to manually identify the faulty line. The Bug Analyzer first understands the problem and identifies the affected file, function and root cause. The Fix Generator then creates a targeted code correction, which is temporarily applied and tested. The system runs the relevant test as well as regression tests before the Reviewer checks the outcome.

The complete flow is:

Bug Report → Analysis → Fix → Patch → Testing → Review → Change Summary

The system therefore does more than just generate code – it also checks whether the proposed change actually works.

Agents & Their Responsibilities

Agent / Component	Role	What it does
Bug Analyzer	Understands the bug	Reads the bug report and code context to identify the affected file, function and root cause.
Fix Generator	Creates the solution	Generates a targeted code fix based on the identified root cause.
Code Patcher	Applies the fix	Temporarily applies the generated change to the affected function while keeping the original code available for restoration.
Test Agent	Validates the fix	Runs the target test and regression tests to check whether the fix works without breaking existing functionality.
Reviewer	Verifies the outcome	Reviews the generated fix and test results to determine whether the reported bug has been resolved.
Change Summary	Documents the change	Provides the affected file, problem, change made, reason and validation result.

Testing & Results

Testing the Generated Fix

To make the system more realistic, we created a small repository containing different types of application logic and intentionally introduced bugs into it. The system was tested on three different bug scenarios:

- Calculator: Empty list causing `ZeroDivisionError`
- Orders: ₹1000 boundary condition missing the discount
- Users: Missing email causing `KeyError`

For the original repository, the test suite showed:

9 passed | 3 failed

After fixing the ₹1000 discount bug:

10 passed | 2 failed

The target boundary test passed, while the remaining failures belonged to the other intentionally introduced bugs.

This allows us to demonstrate that the system does not simply produce a suggested fix, it validates the fix through automated testing.

Technology, Output & Future Scope

Technology Stack & Future Scope

Technology Used

Python – core implementation

Gemini API – bug analysis, code generation and review

pytest – automated unit and regression testing

Gradio – simple user interface

Multi-agent workflow – separates analysis, fixing, testing and review

What the user receives

The final output contains the identified root cause, generated fix, test results, and reviewer assessment

and a change summary explaining what was modified and why.

Future Scope

The current prototype can be extended to larger repositories, more programming languages, stronger sandbox isolation, automatic Git commits/pull requests and more advanced codebase-level bug detection.



Thank You